T.C.
SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

ISE 401 BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ TASARIMI

BSM-SİM: BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ ÖĞRENCİLERİNE YÖNELİK 3D BİLİŞİM İŞ SÜREÇLERİ SİMÜLASYON OYUNU (SERİOUS GAME)

B231200382 – Kaan BİBER

Bölüm	**:**	**BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ**
Danışman	**:**	**Dr.Öğr.Üyesi Burcu ÇARKLI YAVUZ**

2025-2026 Güz Dönemi

ÖNSÖZ

Günümüzde mühendislik eğitimi, teorik bilginin ötesinde pratik deneyim ve disiplinlerarası yetkinlik gerektirmektedir. Bu ihtiyaca yönelik geliştirilen "BSM-SİM" projesi, Bilişim Sistemleri Mühendisliği öğrencilerine sanal bir şirket ortamında güvenli bir deneyim alanı sunan, Unity tabanlı bir "Ciddi Oyun" çalışmasıdır. Proje, teknik bilgi birikimini gerçekçi kriz senaryolarıyla harmanlayarak öğrencilerin mesleki farkındalıklarını mezuniyet öncesinde artırmayı hedeflemektedir.

İÇİNDEKİLER

ÖNSÖZ……..... ii
İÇİNDEKİLER..... iii
ŞEKİLLER LİSTESİ..... v
TABLOLAR LİSTESİ..... vi
ÖZET..... vii

BÖLÜM 1. GİRİŞ..... 1
1.1. Konunun Tanımı ve Önemi 1
1.2. Literatür Özeti ve Arka Plan..... 2
1.3. Amaç ve Hedefler..... 5
1.4. Kullanılan Yöntemler 5
1.4.1. Yazılım geliştirme metodolojisi (yinelemeli yaklaşım)..... 5
1.4.2. Eğitsel içerik geliştirme yöntemi (soru havuzu ve senaryolaştırma) ……6
1.4.3. Oyunlaştırma ve ölçme stratejileri 6
1.5. Raporun Yapısı 7
BÖLÜM 2. SİSTEMATİK YAKLAŞIM 8
2.1. Tasarım Metedolojisi 8
2.2. Matematiksel Model ve Oyun Döngüsü..... 9
2.3. Yazılım Mimarisi..... 11
2.3.1. Veri Katmanı (Data Layer - ScriptableObjects) 11
2.3.2. Yönetim Katmanı (Manager Layer) 11
2.3.3. Kontrol Katmanı (Controller Layer) 12
2.4. Kullanılan Araçlar ve Platform..... 13
BÖLÜM 3. OYUN TASARIMI VE MEKANİKLER..... 14
3.1. Konsept ve Tanıtım..... 15
3.2. Temel Oyun Mekanikleri..... 15
3.2.1. Birinci şahıs kamera kontrolü (FPS) 15
3.2.2. Nesne etkileşim sistemi (raycast interaction)..... 16
3.2.3. Perspektif geçişi (3D - 2D model değişimi)..... 16

3.2.4. Zaman ve gün döngüsü 16
3.3. Görev Ortamları ve Arayüz Tasarımları 17
3.3.1. Sanal bilgisayar (PC) ortamı 17
3.3.2. Donanım ve altyapı ortamı 18
3.3.3. İnsan ve yönetim ortamı 19
3.4. Oyun Ekonomisi ve İlerleme Sistemi 19
3.4.1. Günlük görev döngüsü 20
3.4.2. Kredi (saygınlık) sistemi 21
3.4.3. Kazanma ve kaybetme koşulları 22
3.4.4. Serbest dolaşım (Free-Roam) modu 23
3.5. Bölüm ve Ortam Tasarımı (Level Design) 23
3.6. Eğitsel Geri Bildirim ve Analiz Sistemi 24
BÖLÜM 4. TEST PLANI VE DOĞRULAMA STRATEJİSİ 25
4.1. Fonksiyonel Test Senaryoları (Unit Tests Plan) 25
4.2. Uyumluluk ve Performans Hedefleri 26
4.3. İçerik Geliştirme, Senaryolaştırma ve Doğrulama Süreci 26
4.3.1. Problemlerin Akademik Yeterlilikleri (Müfredat Matrisi) ... 26
4.3.2. Kurumsal ortama uygun senaryolaştırma (bağlamsal öğrenme) 26
4.3.3. Unity entegrasyonu ve oyunlaştırma kalitesi 27
BÖLÜM 5. SONUÇLAR VE ÖNERİLER 28
5.1. Sonuçlar 28
5.2. Öneriler ve Gelecek Çalışmalar 29

KAYNAKLAR 30
EK 31
ÖZGEÇMİŞ 32
BSM 401 BİLGİSAYAR MÜHENDİSLİĞİ TASARIMI DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI 33

ŞEKİLLER LİSTESİ

Figure 1.1 Kolb'un Deneyimsel Öğrenme Döngüsü 3
Figure 3.1 Gemini Pro 2.5 ile hazırlanmış, temsili bir oyun-içi ofis ortamı 14
Figure 3.2 Gemini Pro 2.5 ile hazırlanmış, temsili sanal bilgisayar ortamı 18
Figure 3.3 Gemini Pro 2.5 ile hazırlanmış, temsili donanım-altyapı ortamı 19
Figure 3.4 . Gemini Pro 2.5 ile hazırlanmış, temsili bir görev yönetim arayüzü 20
Figure 3.5 Gemini Pro 2.5 ile hazırlanmış, temsili bir yönetici geri bildirimi 21
Figure 3.6 Gemini PRO 2.5 ile hazırlanmış temsili oyun sonu arayüzü 22
Figure 3.7 Gemini Pro 2.5 ile hazırlanmış, eğitsel geri bildirim sisteminin temsili görseli 24

TABLOLAR LİSTESİ

Table 2.1 Geliştirme Fazları ve Teknik Çıktılar 8
Table 2.2 Kod Bloğu 2.1. Oyun Döngüsü Yöneticisi (C# Unity Prototip) 9
Table 2.3 Kod Bloğu 2.2. Problem Veri Yapısı (C#) 11
Table 2.4. Karakter Kontrol Durum Tablosu 12
Table 4.1 Planlanan Fonksiyonel Test Senaryoları 25

ÖZET

Anahtar kelimeler: Serious Game, Simülasyon, Deneyimsel Öğrenme, Bilişim

Bilişim Sistemleri Mühendisliği (BSM) gibi disiplinlerarası bölümlerde, öğrencilerin teorik bilgi yükünü gerçekçi iş süreçlerine aktarma ve kendilerini sınama konusunda yaşadıkları eksiklikler, bu çalışmanın çıkış noktasını oluşturmaktadır.

Projenin temel amacı, BSM müfredatını kapsayan ve kurumsal senaryolarla zenginleştirilmiş, “BSM-SİM" adlı 3D (low-poly), FPS tabanlı bir "ciddi oyun" (serious game) prototipi geliştirmektir. Ürün geliştirme fazında Unity oyun motoru kullanılarak, "Yinelemeli Yazılım Geliştirme" modeliyle sanal bir şirket ortamı, "Kredi/Gün" döngüsüne dayalı oyun mekanikleri ve modüler (ScriptableObject tabanlı) problem çözme arayüzleri tasarlanacaktır. Yazılım, Donanım ve Yönetim alanlarını içeren en az 20 adet oyunlaştırılmış problem, öğrencilerin hem teknik becerilerini hem de karar alma yetilerini sınayacak şekilde senaryolaştırılacaktır.

Geliştirilen prototipin eğitsel etkinliği, hedef kitle üzerinde yapılacak testler ve nitel analizlerle değerlendirilecektir. Çalışmanın sonucunda, öğrencilerin mesleki yetkinlik farkındalıklarını artıran, pasif öğrenmeyi aktif deneyime dönüştüren yenilikçi bir eğitim aracı ve genişletilebilir bir dijital problem envanteri elde edilecektir.

BÖLÜM 1. GİRİŞ

Bu proje, Bilişim Sistemleri Mühendisliği (BSM) öğrencilerine yönelik, “BSM-SİM” adında, Unity oyun motoru üzerinde geliştirilecek 3D (low-poly modelleme ile) ve FPS (Birinci Şahıs Perspektifi) tabanlı bir "ciddi oyun" (serious game) prototipinin tasarım ve geliştirme süreçlerini kapsamaktadır. Proje, oyuncuyu (öğrenciyi) sanal bir şirket simülasyonuna yerleştirerek; BSM müfredatına dayalı, gerçek hayat problemleri ile senaryolaştırılmış ve oyun içi (in-game) arayüzlerle zenginleştirilmiş bir problem setini tamamlamasını hedefler.

1.1. Konunun Tanımı ve Önemi

Bilişim Sistemleri Mühendisliği (BSM) gibi disiplinlerarası programlar, öğrencilere yazılım geliştirme, veritabanı yönetimi, sistem analizi ve iş yönetimi gibi çok geniş bir yelpazede teorik bilgi sunmaktadır. Ancak mevcut eğitim literatüründe, bu farklı disiplinlerin kurumsal bir yapı içerisinde nasıl entegre çalıştığını öğrencilerin deneyimlemesi ve teorik bilgilerini gerçekçi kriz senaryolarında sınaması noktasında önemli bir eksiklik bulunmaktadır. Bu durum, öğrencilerin teorik bilgi yükünü iş hayatında nasıl değerlendirecekleri ve hangi alana (Yazılım, Donanım veya Yönetim) yatkın oldukları konusunda farkındalık geliştiremeden mezun olmalarına yol açmaktadır.

Bu tasarım çalışmasında, söz konusu eksikliği gidermek amacıyla salt bir "eğitici materyal" sunmak yerine, "oyunlaştırılmış bir problem seti" sunan yenilikçi bir yaklaşım benimsenmiştir. Proje, öğretici rolünün yanı sıra, öğrenciye gerçekçi bir sınama imkânı tanıyan bir "ciddi oyun" olarak tasarlanmıştır. Akademik bir kaynaktan "doğru SQL sorgusunun" ne olduğunu okumak ile; bir şirket simülasyonunda, zaman baskısı altında ve departmanların çelişen talepleri (Örn: Muhasebe departmanının acil veri isteği) arasında "yeterince iyi" olan sorguyu yazmak arasında büyük bir bilişsel fark vardır. Projenin bir vaka analizi seti değil, bir

"oyun" olmasının temel nedeni budur. Proje, öğrenciyi pasif bir "okuyucu" rolünden, aktif bir "karar alıcı" (Mühendis) rolüne taşımaktadır.

Öğrenci bu süreçte sadece doğru cevabı bilmek zorunda değil; aynı zamanda karar vermek, uygulamak ve eylemlerinin sistemsel sonuçlarını (kredi kaybı, performans uyarısı, terfi) anında görmek durumundadır. Oyun formatının getirdiği motivasyon faktörü, bu zorlu "sınama" sürecini sürdürülebilir kılmayı amaçlar. Böylece araştırma, pasif metotların yetersiz kaldığı "teorik bilginin pratiğe dökülmesi" sorununu, aktif ve sonuç odaklı bir simülasyon modeli ile çözmeyi hedeflemektedir.

1.2. Literatür Özeti ve Arka Plan

Bu çalışmanın anlaşılabilmesi için öncelikle "Ciddi Oyunlar" kavramı ele alınmalıdır. Michael ve Chen (2005), ciddi oyunları temel amacı sadece eğlence olmayan; kullanıcıya eğitim, alıştırma ve bilgilendirme gibi somut çıktılar sunarak belirli bir beceri veya tutum kazandırmayı hedefleyen oyunlar olarak tanımlamaktadır [4].

Literatür incelendiğinde, mühendislik eğitiminde simülasyonların öneminin giderek arttığı görülmektedir. Negahban, mühendislik eğitiminde fiziksel deneyimlemenin yerini giderek dijital ve kapsayıcı simülasyon ortamlarının (immersive simulated environments) aldığını ve bu geçişin karmaşık mühendislik problemlerinin çözümünde kritik bir rol oynadığını belirtmektedir [5]. Benzer şekilde, De Gloria ve arkadaşları, ciddi oyunların karmaşık ve disiplinlerarası konuları öğretmede, bağlamsal öğrenmeyi destekleyerek geleneksel pasif yöntemlerden daha etkili olduğunu ortaya koymuştur [2]. Ancak bu etkinin sağlanması, rastgele bir oyunlaştırmadan ziyade; Arnab ve arkadaşlarının vurguladığı gibi, öğrenme hedefleri ile oyun mekaniklerinin (Learning Mechanics - Game Mechanics) doğru bir şekilde eşleştirilmesine bağlıdır [1].

Ayrıca Kolb'un (1984) "Deneyimsel Öğrenme Teorisi", bireylerin en iyi "deneyimleyerek ve yansıtarak" öğrendiğini savunmaktadır [3].

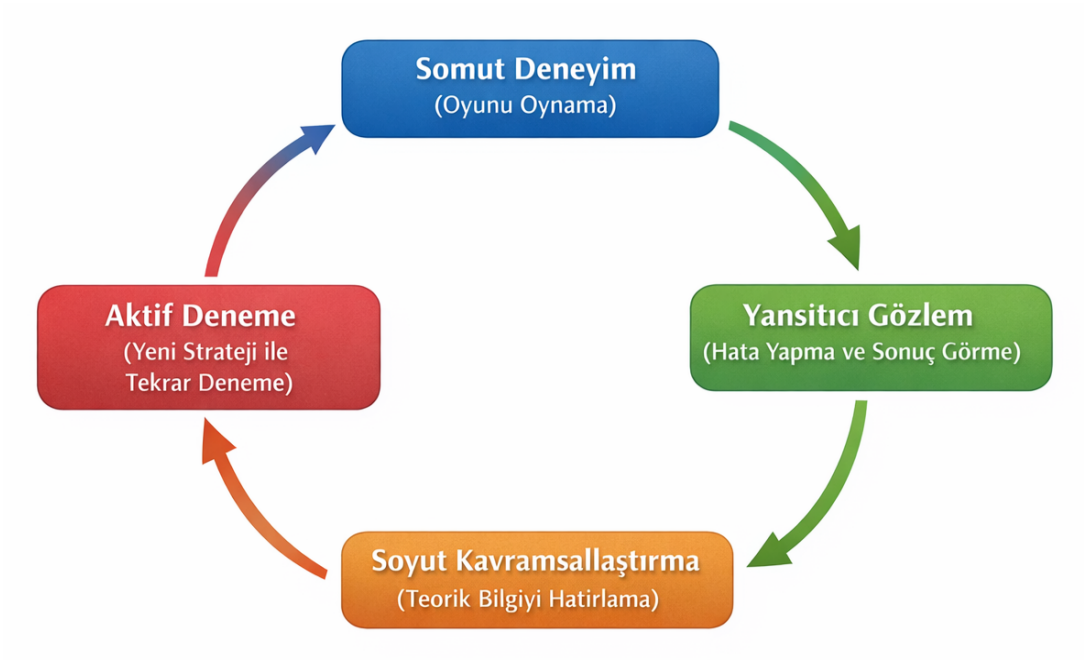

Figure 1.1 Kolb'un Deneyimsel Öğrenme Döngüsü

Bu teorik altyapıyı dijital ortama aktarırken, Akpınar ve Kutbay'ın önerdiği üzere, öğrencinin serbestçe keşfetmesine olanak tanıyan ancak gerektiğinde doğru yönlendirmeleri sağlayan öğrenme desteklerinin (learning supports) tasarıma dahil edilmesi gerekmektedir [6].

Mevcut çalışmalar genellikle tekil konulara (sadece kodlama veya sadece ağ yönetimi) odaklanırken; bu çalışma, BSM'nin disiplinlerarası yapısını (teknik ve yönetimsel) bütüncül bir şirket simülasyonu içinde ele alarak literatürdeki bu boşluğu doldurmayı hedeflemektedir.

1.3. Amaç ve Hedefler

Bu çalışmanın ana amacı, Bilişim Sistemleri Mühendisliği lisans öğrencilerinin teorik müfredat boyunca edindikleri pasif bilgiyi, aktif bir şekilde pratiğe dökebilecekleri bir ortam yaratarak "teori" ve "uygulama" arasındaki boşluğu kapatmaktır. Proje, öğrencilerin bilgi birikimlerini gerçekçi kurumsal senaryolarla harmanlanmış, oyunlaştırılmış bir problem seti aracılığıyla sınamalarını sağlamayı hedeflemektedir.

Bu amacı gerçekleştirmek için, Unity oyun motoru üzerinde çalışacak 3D ve FPS tabanlı, eğitici bir "ciddi oyun" prototipi geliştirilecektir. Çalışmanın birincil çıktısı bu teknik prototip olmakla birlikte; ikincil ve uzun vadeli amacı, geliştirilen bu prototipin bir eğitim aracı olarak etkinliğini değerlendirmektir. Bu değerlendirme, oyunun öğrencilerin kendi teknik ve yönetsel yetkinliklerindeki eksikliklere dair farkındalıklarını artırma başarısını analiz etmeyi kapsayacaktır.

1.4. Kullanılan Yöntemler

Bu tasarım çalışmasında, projenin hem teknik bir yazılım ürünü hem de eğitsel bir materyal olması sebebiyle hibrit bir metodoloji izlenmiştir. Süreç; yazılım geliştirme, eğitsel içerik tasarımı ve oyunlaştırma stratejileri olmak üzere üç ana eksende yürütülmektedir.

1.4.1. Yazılım geliştirme metodolojisi (yinelemeli yaklaşım)

Oyun geliştirme süreçlerinin doğası gereği, sürekli test ve geri bildirim döngüsüne ihtiyaç duyulması nedeniyle "Yinelemeli (Iterative) Yazılım Geliştirme" modeli tercih edilmiştir. Bu yöntemle proje; analiz, tasarım, prototipleme ve test aşamalarından oluşan döngüler halinde ilerletilmektedir.

- Tasarım: Projenin sınırlarını ve mekaniklerini belirleyen GDD (Oyun Tasarım Dokümanı) hazırlanmıştır.

- Veri Mimarisi: Oyunun genişletilebilir olması hedeflendiğinden, "Veri Güdümlü Tasarım" (Data-Driven Design) ilkesi benimsenmiştir. Bu kapsamda, kod tekrarını önlemek ve içeriği dinamik hale getirmek için ScriptableObject tabanlı modüler bir yapı kurgulanmıştır.

1.4.2. Eğitsel içerik geliştirme yöntemi (soru havuzu ve senaryolaştırma)

Oyunun "ciddi oyun" (serious game) niteliğini sağlamak amacıyla, içerik üretiminde "Müfredat Tabanlı Senaryolaştırma" yöntemi kullanılmıştır. Bilişim Sistemleri Mühendisliği (BSM) lisans müfredatı taranarak; Yazılım, Donanım/Altyapı ve Yönetim olmak üzere üç ana kategoride bir "Akademik Problem Havuzu" oluşturulmuştur. Başlangıç aşamasında bu havuzdan en az 20 adet kazanım odaklı problem seçilmiştir. Seçilen akademik sorular, doğrudan sınav sorusu gibi sunulmak yerine; kurumsal bir şirkette yaşanabilecek gerçekçi krizlere (Örn: "Maaş veritabanı çöktü", "Sunucu odasında aşırı ısınma var") dönüştürülerek senaryolaştırılmıştır.

1.4.3. Oyunlaştırma ve ölçme stratejileri

Öğrencilerin motivasyonunu artırmak ve deneyimsel öğrenmeyi sağlamak için "Akış (Flow) Teorisi" temel alınmıştır. Oyuncunun başarısı, statik puanlar yerine dinamik bir "Şirket İtibarı / Kredi" sistemi üzerinden simüle edilmektedir. "Günlük Görev Döngüsü" kurgusuyla, oyuncunun zaman baskısı altında karar verme yetisi sınanmaktadır.

Oyuncunun (öğrencinin) performansını analiz eden algoritmik bir geri bildirim yöntemi tasarlanmıştır. Bu sistem, oyuncunun başarısız olduğu görevleri takip ederek, spesifik teknik eksiklerini (Örn: "SQL Join komutlarında hata yapıyorsunuz") saptamak ve ilgili akademik kaynağa yönlendirmek üzere kurgulanmıştır.

1.5. Raporun Yapısı

Bu tasarım projesi raporu, okuyucunun projeyi mühendislik temellerinden başlayıp son ürün tasarımına ve test stratejilerine kadar adım adım takip edebilmesi için şu şekilde yapılandırılmıştır:

- Bölüm 2 (Sistematik Yaklaşım): Projenin teknik iskeletini oluşturan yazılım mimarisi, veri yapıları (ScriptableObject), kullanılan geliştirme metodolojileri ve oyunun karar mekanizmasını yöneten matematiksel modeller (Kredi/Gün formülasyonu) detaylandırılmıştır.
- Bölüm 3 (Oyun Tasarımı ve Mekanikler): Teknik altyapı üzerine kurulan oyun konsepti, senaryo kurgusu, oyuncu etkileşim mekanikleri (FPS, Raycast vb.), arayüz tasarımları ve oyun ekonomisi açıklanmıştır.
- Bölüm 4 (Test Planı ve Doğrulama Stratejisi): Tasarlanan sistemin uygulama fazında tabi tutulacağı fonksiyonel test senaryoları, algoritmaların teorik doğrulaması (Dry Run) ve eğitsel içeriğin akademik validasyon süreçleri sunulmuştur.
- Bölüm 5 (Sonuçlar ve Öneriler): Tasarım çalışmasından elde edilen teknik çıktılar, prototipin mevcut durumu ve projenin ikinci fazı için planlanan saha testleri ile geliştirme önerileri tartışılmıştır.

BÖLÜM 2. SİSTEMATİK YAKLAŞIM

"BSM-SİM" projesi, sadece görsel bir tasarım çalışması değil, aynı zamanda ölçeklenebilir ve sürdürülebilir bir yazılım mimarisi üzerine kurgulanmış mühendislik tabanlı bir sistemdir. Bu bölümde, projenin hayata geçirilmesinde izlenen sistematik yol haritası ve teknik altyapı detaylandırılmıştır. Simülasyonun geliştirilme sürecinde benimsenen yazılım metodolojileri, oyunun karar mekanizmasını yöneten matematiksel modeller, veri mimarisi (Data Architecture) ve kullanılan teknolojik araçlar sistematik bir bütünlük içinde sunulacaktır. Bu yaklaşım, sistemin hem teknik kararlılığını (stability) hem de içeriğin ileride kolayca genişletilebilmesini (scalability) garanti altına almayı hedeflemektedir.

2.1. Tasarım Metedolojisi

Projenin geliştirilme sürecinde, oyun geliştirme süreçlerinin iteratif (yinelemeli) doğasına uygun olarak "Yinelemeli Yazılım Geliştirme Modeli" (Iterative Development Model) benimsenmiştir. Bu modelde proje, her biri çalışan bir ürün parçası ortaya koyan üç ana faza bölünmüştür. Fazların teknik çıktıları Tablo 2.1'de özetlenmiştir.

Table 2.1 Geliştirme Fazları ve Teknik Çıktılar

Faz	Odak Alanı	Teknik Çıktı (Deliverable)
Faz - 1	Çekirdek Mekanikler	FPS Kontrolcüsü (PlayerController.cs), Fizik Motoru Ayarları, Ofis Sahnesi (Mesh)
Faz - 2	Etkileşim ve Arayüz	Raycast Sistemi, Durum Makinesi (GameStates), Dinamik UI Panelleri.
Faz - 3	İçerik ve Döngü	Raycast Sistemi, Durum Makinesi (GameStates), Dinamik UI Panelleri.

2.2. Matematiksel Model ve Oyun Döngüsü

Sistemin ana akışı, oyuncunun performansına göre değişen dinamik bir "Kredi Tabanlı Durum Makinesi" üzerine kuruludur. Oyun döngüsü, her oyun günü sonunda hesaplanan kümülatif saygınlık puanı üzerinden modellenmiştir. Oyunun durumunu belirleyen transfer fonksiyonu (Denklem 2.1) aşağıda verilmiştir.

$$S_{t+1} = S_t + \sum_{i=1}^{N}(G_i \times \alpha) - \sum_{j=1}^{M}\left(H_j \times \beta\right) - C_{sabit} \tag{2.1}$$

Parametreler:

- $\boldsymbol{S_t}$ **:** Güne başlarken sahip olunan kredi.
- $\boldsymbol{G_i}$ **:** Tamamlanan görevde kazanılan taban puan.
- $\boldsymbol{\alpha}$ **:** Görev zorluk katsayısı (1.0 - 1.5 arası).
- $\boldsymbol{H_j}$ **:** Yapılan hata sayısı.
- $\boldsymbol{\beta}$ **:** Hata ceza katsayısı (Örn: 10 puan).
- $\boldsymbol{C_{sabit}}$ **:** Günlük zorunlu operasyonel maliyet.

Bu matematiksel modelin yazılım tarafındaki algoritma akışı (Pseudo-Code) ise Kod Bloğu 2.1'de gösterilmiştir.

Table 2.2 Kod Bloğu 2.1. Oyun Döngüsü Yöneticisi (C# Unity Prototip)

```
using UnityEngine;
using System.Collections;

public class GameManager : MonoBehaviour
{
    [Header("Oyun Ekonomisi")]
    public float currentCredit = 100f; // Parametre: St (Başlangıç Kredisi)
    public int dayCount = 1;
    private bool isGameActive = true;
```

```
    private IEnumerator Start()
    {
        while (isGameActive)
        {
            // Gün döngüsünü başlat ve tamamlanmasını bekle
            yield return StartCoroutine(RunDayCycle());

            // Gün sonu puan ve kredi hesaplaması (Bkz: Denklem 2.1)
            EvaluateDayPerformance();

            if (CheckGameEndConditions()) break;

            dayCount++;
            yield return new WaitForSeconds(2f); // Gün geçiş animasyonu süresi
        }
    }

    private IEnumerator RunDayCycle()
    {
        Debug.Log($"Gün {dayCount} Başladı.");

        // ScriptableObject veritabanından o güne ait görevler yüklenir
        float dailyTimer = 300f; // 5 Dakika (Simüle Edilen İş Günü)

        while (dailyTimer > 0)
        {
            dailyTimer -= Time.deltaTime;
            // Oyuncu girdileri ve görev tamamlanma durumları dinlenir
            yield return null;
        }
    }

    private bool CheckGameEndConditions()
    {
        if (currentCredit <= 0) return true; // KOVULMA (Fail State)
        if (dayCount >= 30) return true;     // TERFİ (Win State)
        return false;
    }
}
```

2.3. Yazılım Mimarisi

"BSM-SİM", Unity oyun motorunun Bileşen Tabanlı (Component-Based) yapısı üzerine inşa edilmiş, modüler bir C# mimarisine sahiptir. Sistem üç ana katmanda kurgulanmıştır.

2.3.1. Veri Katmanı (Data Layer - ScriptableObjects)

Oyunun "genişletilebilir" olması hedeflendiğinden, görevler ve problemler kod içine gömülmek yerine (Hard-coded), Unity'nin ScriptableObject varlıkları olarak dışsallaştırılmıştır. Bu yapı, veri ile mantığı birbirinden ayırır. Kullanılan veri sınıfının yapısı Kod Bloğu 2.2'de sunulmuştur.

Table 2.3 Kod Bloğu 2.2. Problem Veri Yapısı (C#)

```
[CreateAssetMenu(fileName = "YeniProblem", menuName = "BSM/Problem")]
public class ProblemData : ScriptableObject
{
    public string problemID;        // Örn: "SQL_01"
    [TextArea] public string soruMetni;
    public string dogruCevap;       // SQL sorgusu veya kod çıktısı
    public int zorlukSeviyesi;      // 1-5 arası
    public string ipucuMetni;
    public DersTuru dersKategorisi;  // Enum: Yazılım, Donanım, Yönetim
}
```

Bu mimari sayesinde, derleme (compile) süresine gerek kalmadan Unity Editör üzerinden sağ tıklanarak yüzlerce yeni problem dosyası .asset formatında oluşturulabilmektedir.

2.3.2. Yönetim Katmanı (Manager Layer)

Oyunun genel akışını kontrol eden sınıflar, bellekte tek bir örneğinin bulunmasını garanti eden Singleton Tasarım Deseni ile kurgulanmıştır. GameManager, oyunun durumunu (Başladı, Bitti, Duraklatıldı), gün döngüsünü ve kredi hesaplamalarını yönetir. InteractionManager, oyuncunun baktığı nesneyi (Raycast ile) algılar. UIManager, observer (Gözlemci) deseni ile veri değişikliklerini dinler ve arayüzü günceller.

2.3.3. Kontrol Katmanı (Controller Layer)

Sistem, oyuncunun etkileşim durumuna göre iki farklı durum (State) arasında geçiş yapan bir "Sonlu Durum Makinesi" (Finite State Machine) kullanır. Bu geçişler Tablo 2.2'de gösterilmiştir.

Table 2.4. Karakter Kontrol Durum Tablosu

Durum (State)	Tetikleyici (Trigger)	Aktif Girdiler	Kamera Davranışı
FPS Modu	Oyun Başlangıcı / ESC Tuşu	WASD (Yürüme), Mouse (Bakış)	Serbest Bakış
Terminal Modu (Yazılım)	Ofis Masasına Oturma (E)	Klavye (Kodlama), Mouse (UI)	Monitöre Kilitli
Tamir Modu (Donanım)	Sunucu Kabinine Yaklaşma (E)	Mouse (Kablo Takma / Switch Açma)	Panele Zoom / İzometrik
Diyalog Modu (Yönetim)	NPC ile Konuşma (E)	Mouse (Cevap Seçimi), Space (İlerle)	Sinematik / Karakter Odaklı

2.4. Kullanılan Araçlar ve Platform

Projenin geliştirilme ve test süreçlerinde kullanılan teknolojiler ve versiyon bilgileri aşağıdadır:

Oyun Motoru: Unity 2022.3 LTS (Render Pipeline: URP)
Dil & IDE**:** C# / Microsoft Visual Studio 2022
3D Modelleme**:** Blender 3.6 (Low-Poly Asset Üretimi)
Veri Formatı**:** JSON / Asset Files
Hedef Platform: Windows 10/11 (x64 Mimarisi, DirectX 11 destekli GPU)

BÖLÜM 3. OYUN TASARIMI VE MEKANİKLER

"BSM-SİM", Bilişim Sistemleri Mühendisliği öğrencilerinin teorik bilgilerini sanal bir iş ortamında deneyimlemeleri amacıyla tasarlanmış, Birinci Şahıs Kamera Açısına (FPS) sahip bir 3D simülasyon oyunudur. Oyuncu, kurgusal bir teknoloji şirketinde işe yeni başlamış "Junior Bilişim Mühendisi" rolünü üstlenmektedir.

Figure 3.1 Gemini Pro 2.5 ile hazırlanmış, temsili bir oyun-içi ofis ortamı

3.1. Konsept ve Tanıtım

Oyunun temel amacı; şirketin Yazılım, Ağ Yönetimi ve İnsan Kaynakları gibi farklı departmanlarından gelen günlük görevleri, talepleri ve krizleri çözerek iş gününü başarıyla tamamlayıp yeni bir güne geçmektir. Oyuncu, BSM müfredatında edindiği teorik altyapıyı pratiğe dönüştürürken, şirketteki "Saygınlık Kredisini" (Reputation Credit) korumalı ve nihayetinde bütün oyun günlerini kredisi bitmeden tamamlayarak oyunu bitirmelidir.

Bu süreçte proje, klasik didaktik öğretim araçlarından ayrışarak; oyuncuyu zaman baskısı ve karar verme stresi altında bırakmayı ve bu sayede "akışta öğrenme" (flow learning) deneyimi sunmayı hedeflemektedir. Simülasyon ortamı, öğrencinin hangi konularda yetkin olduğunu veya hangi alanlarda (Yazılım, Donanım veya Yönetim) eksiklik yaşadığını keşfetmesini sağlayacak bir "yetkinlik farkındalığı" aracı olarak kurgulanmıştır. Ayrıca, 4 yıllık lisans müfredatına referans veren ve kurumsal ortama uygun senaryolaştırılmış bu problemlerin çözümü sırasında, oyuncunun performansını analiz eden ve kişiselleştirilmiş dönütler sunan bir "Eğitsel Geri Bildirim Sistemi" tasarıma entegre edilmiştir.

3.2. Temel Oyun Mekanikleri

Oyunun etkileşim tasarımı, oyuncunun sanal ortamda bir "gözlemci" değil, aktif bir "uygulayıcı" olması prensibi üzerine kurulmuştur. Oyuncunun simülasyon dünyasıyla kurduğu ilişkiyi tanımlayan temel mekanikler aşağıda detaylandırılmıştır:

3.2.1. Birinci şahıs kamera kontrolü (FPS)

Proje, oyuncunun kendini "Junior Mühendis" karakteriyle özdeşleştirmesini (immersion) kolaylaştırmak amacıyla Birinci Şahıs Kamera açısı (First Person Perspective) ile tasarlanmıştır.

Hareket etmek için oyuncu, ofis ortamında, sunucu odasında ve üretim alanlarında standart klavye kontrolleri (WASD) ile serbestçe yürüyebilir.
Etrafa bakış için fare (Mouse) hareketleri ile X ve Y eksenlerinde serbest bakış açısı kontrol edilir. Kamera yüksekliği, ortalama bir insan boyuna sabitlenerek gerçekçi bir perspektif sağlanmıştır.

3.2.2. Nesne etkileşim sistemi (raycast interaction)

Oyuncu ile dünyadaki nesneler (Bilgisayarlar, Sunucu Kabinleri, Kapılar, NPC'ler) arasındaki etkileşim, "Baktığını Seç" mantığına dayalı Işın İzleme (Raycasting) teknolojisi ile kurgulanmıştır. Algılama için oyuncu; bir nesneye yaklaşıp baktığında, eğer nesne etkileşime uygunsa (Örn: Bir Switch cihazı), ekranın ortasındaki imleç (Crosshair) şekil değiştirir ve oyuncuya görsel bir ipucu verir. Tetikleme ise etkileşim tuşuna basıldığında, nesnenin türüne göre ilgili olay (Kapının açılması, Diyalog penceresinin gelmesi vb.) gerçekleşir.

3.2.3. Perspektif geçişi (3D - 2D model değişimi)

Oyun, iki farklı kontrol şeması arasında dinamik geçiş yapmaktadır:

- Serbest Dolaşım (3D): Oyuncu ofis içinde gezerken FPS kontrollerini kullanır.
- Odaklanmış Çalışma (2D): Oyuncu masasına oturup sanal bilgisayarı açtığında veya bir sunucu paneline yaklaştığında, kamera sabitlenir ve kontrol "Mouse İmleci" moduna geçer. Bu mekanik, oyuncunun karmaşık mühendislik arayüzlerini (Terminal, Kod Editörü) rahatça kullanabilmesini sağlar. "Çıkış" tuşu ile tekrar 3D dolaşım moduna dönülür.

3.2.4. Zaman ve gün döngüsü

Oyun, gerçek zamanlı sayaçların her görev için ayrı ayrı çalıştığı bir yapı yerine, bütüncül bir "İş Günü" simülasyonu üzerine kuruludur. İş günü, her bölüm bir "İş Günü"nü temsil eder. Gün başladığında oyuncuya çözmesi gereken problem listesi iletilir. Zaman Yönetimi için oyuncu, mesai saati dolmadan veya kendisine verilen

süre içerisinde, ofis içinde fiziksel olarak hareket edip ilgili departmanlara giderek problemleri çözmek zorundadır.

3.3. Görev Ortamları ve Arayüz Tasarımları

Oyunun temel oynanış döngüsü, oyuncunun problemin türüne göre üç farklı sanal ortamda (PC, Donanım, İnsan) etkileşime girmesi üzerine kurgulanmıştır. Her ortam, kendine özgü arayüz (UI) setlerine ve etkileşim mekaniklerine sahiptir. Tasarımda "Sınırlı Mekanik, Değişen İçerik" prensibi benimsenmiş; böylece aynı arayüz üzerinden farklı parametreler girilerek birçok farklı mühendislik problemi senaryolaştırılabilir hale getirilmiştir. Modüler yapı sayesinde oyuna ileride yeni problemler eklemek de kolaylaşacaktır.

3.3.1. Sanal bilgisayar (PC) ortamı

- Oyuncu, ofis içerisindeki masasına oturup bilgisayarı ile etkileşime geçtiğinde oyun kamerası monitöre odaklanmakta ve kontrol şeması 3D karakter kontrolünden 2D imleç (mouse) - klavye kontrolüne geçmektedir. Bu ortamda çözülecek problem tiplerinden bazılarına örnek olarak şunlar verilebilir:
- Yazılım Sorunları: Oyuncudan, verilen bir senaryoya uygun kod parçacığını tamamlaması veya hatalı bir algoritmayı düzeltmesi istenir. (Örn: C# ile yazılmış hatalı bir döngüyü düzeltmek).
- Veri Yönetimi (SQL): Şirketin satış veya stok verilerine dair raporlar istendiğinde, oyuncunun doğru SQL sorgusunu (Select, Join vb.) yazarak ilgili veriyi çekmesi gereken arayüzdür.
- Muhasebe ve Satın Alma**:** Şirket bütçesini etkileyen dijital faturaların onaylanması veya reddedilmesi süreçlerini içerir.

Figure 3.2 Gemini Pro 2.5 ile hazırlanmış, temsili sanal bilgisayar ortamı

3.3.2. Donanım ve altyapı ortamı

Sadece masa başında değil, fiziksel müdahale gerektiren problemlerde oyuncunun ilgili lokasyona (Sunucu Odası, Fabrika Bandı vb.) gitmesi gerekmektedir.

- Sunucu Odası (Server Room): Ağ problemlerinde oyuncu sunucu kabinleri (Rack) ile etkileşime girer. Doğru portlara kablo takma (Patching), ağ anahtarlarını (Switch) konfigüre etme veya arızalı diskleri değiştirme gibi eylemler, tıklanabilir 3D arayüz elemanları ile simüle edilir.
- Üretim Bandı**:** Fabrika veya depo kısmındaki otomasyon hatalarında, oyuncunun doğru şalterleri indirmesi veya sensör ayarlarını yapması gereken panellerdir.

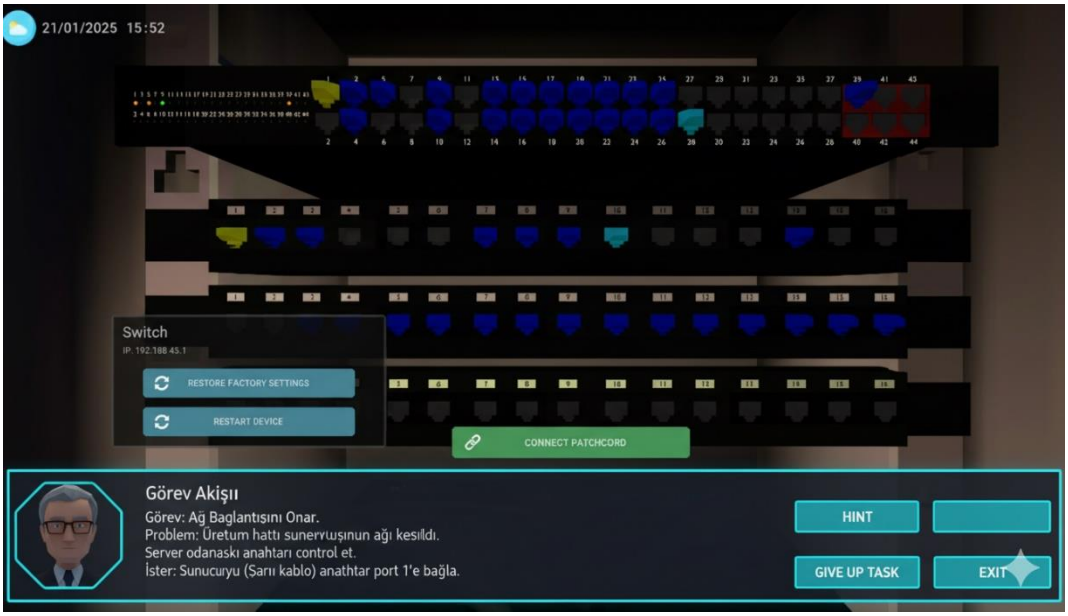

Figure 3.3 Gemini Pro 2.5 ile hazırlanmış, temsili donanım-altyapı ortamı

3.3.3. İnsan ve yönetim ortamı

Bilişim Sistemleri Mühendisliğinin "Yönetim" ayağını simüle eden bu ortamda, oyuncu ofis içindeki diğer karakterlerle (NPC) diyaloğa girer. Diyalog Sistemi şu şekilde işler, insan Kaynakları (İK) krizleri, yönetici talepleri veya ekip içi çatışmalarda oyuncuya çoktan seçmeli diyalog ağaçları sunulur. Oyuncunun seçeceği cevap (Örn: "Agresif", "Uzlaşmacı" veya "Prosedürel"), krizin sonucunu ve şirketteki itibarını doğrudan etkiler.

3.4. Oyun Ekonomisi ve İlerleme Sistemi

Oyunun ilerleme ve kazanma/kaybetme koşulları, "Günlük Döngü" ve "Kredi Sistemi" üzerine kuruludur. Bu yapı, oyuncuyu sürekli bir kaynak yönetimi ve performans baskısı altında tutarak "akışta öğrenme" (flow learning) deneyimini destekler.

3.4.1. Günlük görev döngüsü

Oyun, ardışık günlerden oluşan bir takvim sistemine sahiptir. Her oyun gününde oyuncuya, zorluk seviyesi giderek artan belirli sayıda problem (görev) atanır. Oyuncu, kendisine atanan tüm problemleri "Olumlu" (Çözüldü) veya "Olumsuz" (Çözülemedi/Hatalı) olarak sonuçlandırana kadar gün bitmez. Tüm görevler sonuçlandığında gün sonu raporu ekranı gelir ve bir sonraki güne geçilir.

Figure 3.4 . Gemini Pro 2.5 ile hazırlanmış, temsili bir görev yönetim arayüzü

3.4.2. Kredi (saygınlık) sistemi

Oyuncu oyuna belirli bir başlangıç kredisi (Saygınlık Puanı) ile başlar. Bu kredi, oyuncunun şirketteki varlığını temsil eder.

- Kazanım: Başarıyla tamamlanan her teknik ve yönetimsel görev, görevin zorluğuna göre oyuncunun kredisini artırır.
- Kayıp: Hatalı çözümler, süresi geçirilen görevler veya yanlış yönetimsel kararlar kredi puanının düşmesine neden olur.
- Yükseltme (Upgrade) Mekaniği: Oyuncu, biriktirdiği kredileri ilerleyen günlerde işini kolaylaştıracak araçları (Örn: Daha hızlı derleyici, otomatik hata ayıklayıcı veya ipucu sistemi) satın almak için harcayabilir. Bu mekanik, oyuncuya stratejik bir kaynak yönetimi katmanı ekler.

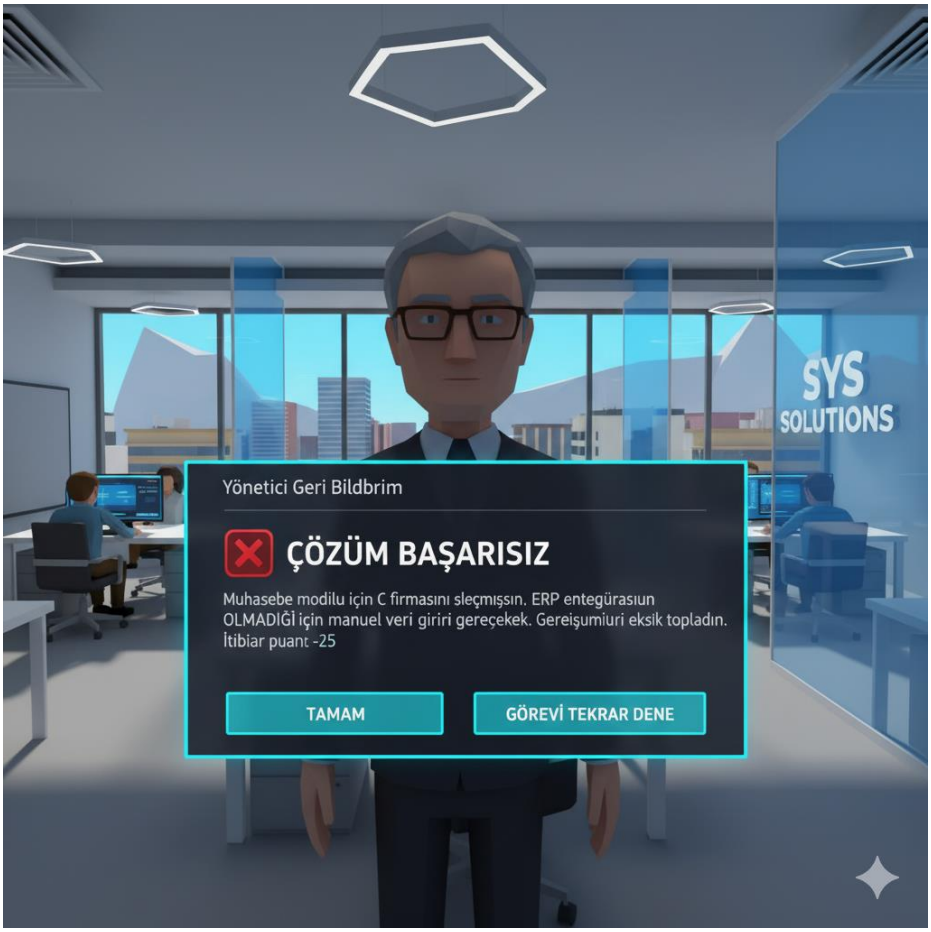

Figure 3.5 Gemini Pro 2.5 ile hazırlanmış, temsili bir yönetici geri bildirimi

3.4.3. Kazanma ve kaybetme koşulları

- Oyun Sonu (Game Over): Herhangi bir günde kredisi sıfıra (0) düşen oyuncu, yetersiz performans gerekçesiyle şirketten kovulur ve oyun başarısızlıkla sonuçlanır.
- Zafer (Win): Belirlenen gün sayısını (Örn: 30 Gün) kredisi tükenmeden tamamlayan oyuncu, "Terfi" alarak oyunu başarıyla bitirir.

Figure 3.6 Gemini PRO 2.5 ile hazırlanmış temsili oyun sonu arayüzü

3.4.4. Serbest dolaşım (Free-Roam) modu

Oyunun ana kurgusundaki "Günlük Döngü" ve "Kredi" baskısından bağımsız olarak, öğrencilerin simülasyon ortamını kendi hızlarında ve stressiz bir şekilde deneyimlemelerine olanak tanıyan "Serbest Dolaşım" (Free-Roam / Sandbox) modu tasarıma eklenmiştir. Bu modun temel özellikleri şunlardır:

- Baskısız Ortam: "Kredi/Saygınlık" sistemi ve zaman sınırları devre dışı bırakılır. Hatalı çözümler veya gecikmeler oyuncunun "Game Over" olmasına (işten çıkarılmasına) neden olmaz.
- Keşfedici Öğrenme: Amaç, simülasyonu bir "meydan okuma" (challenge) aracından çıkarıp, saf bir "etkileşimli problem seti"ne dönüştürmektir. Oyuncu, ofis, sunucu odası ve üretim bandı gibi tüm departmanlara serbestçe erişebilir.
- Tekrar Edilebilirlik: Oyuncu, zorlandığı bir teknik problemi, kredi kaybetme korkusu olmadan defalarca deneyebilir ve farklı çözüm yollarını araştırabilir. Bu mod, özellikle konuyu yeni öğrenen ve "sınanmak" yerine "keşfetmek" isteyen kullanıcılar için rahat bir çalışma alanı sunmayı hedefler.

3.5. Bölüm ve Ortam Tasarımı (Level Design)

Oyun dünyası, bir teknoloji şirketinin işleyişini yansıtacak şekilde 3D olarak modellenmiştir. Oyuncu bu ortamda serbestçe gezinebilir. Ortam üç ana bölgeye ayrılmıştır:

- İdari Ofis**:** Oyuncunun kendi masasının, yöneticilerin ve İK departmanının bulunduğu, "Sanal PC" ve "İnsan" görevlerinin gerçekleştiği alandır.
- Sistem Odası (Server Room): Ağ ve altyapı sorunlarının çözüldüğü, sunucu kabinlerinin yer aldığı teknik alandır.
- Fabrika/Depo: Üretim bandı ve lojistik ile ilgili "Donanım" problemlerinin çözüldüğü endüstriyel alandır.

3.6. Eğitsel Geri Bildirim ve Analiz Sistemi

Projenin birincil amacı olan "eğitim" fonksiyonu, oyun içi geri bildirim mekanizması ile sağlanmaktadır. Oyuncunun sanal bilgisayarında "Görev Geçmişi" (Log) adında bir uygulama bulunur. Performans analizi için Oyuncu geçmiş günlerde tamamladığı veya başarısız olduğu görevleri buradan inceleyebilir. Bunun yanı sıra akademik bir eşleştirme de söz konusudur. Sistem, oyuncunun başarısız olduğu görevleri analiz ederek, oyuncunun eksik olduğu teorik konuyu tespit eder.

Örnek Senaryo: Oyuncu bir yazılım görevinde "Event" yapısını yanlış kurguladıysa, sistem geri bildirim olarak; "Başarısız: Kod mimarisinde hata. C# Delegates ve Events konusunda eksiğin var. Şu kaynağı incelemen önerilir: [Kitap/Makale Adı]" şeklinde akademik bir yönlendirme yapar. Bu özellik, oyunun sadece bir simülasyon değil, aynı zamanda kişiselleştirilmiş bir öğrenme aracı olmasını sağlar.

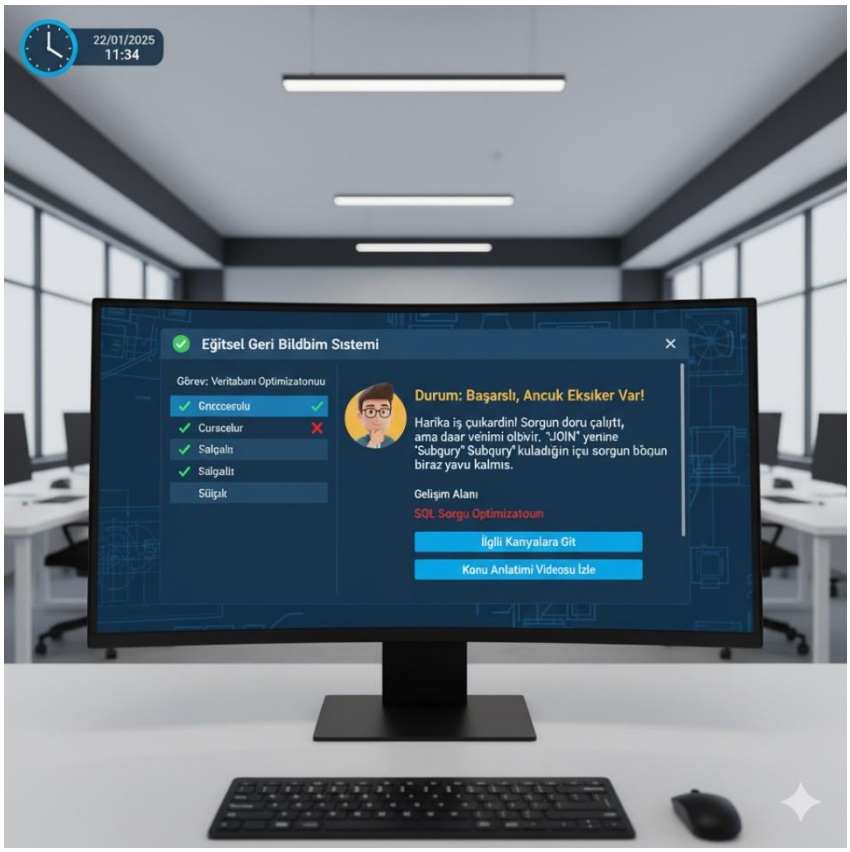

Figure 3.7 Gemini Pro 2.5 ile hazırlanmış, eğitsel geri bildirim sisteminin temsili görseli

BÖLÜM 4. TEST PLANI VE DOĞRULAMA STRATEJİSİ

Bu bölümde, tasarım aşaması tamamlanan "BSM-SİM" projesinin, uygulama (implementasyon) fazında tabi tutulacağı test süreçleri, doğrulama kriterleri ve beklenen sonuç senaryoları detaylandırılmıştır. Test stratejisi; teknik işlevsellik (Verification) ve eğitsel uygunluk (Validation) olmak üzere iki ana eksende kurgulanmıştır.

4.1. Fonksiyonel Test Senaryoları (Unit Tests Plan)

Oyunun mekaniklerinin doğru çalıştığını doğrulamak için Unity Test Runner altyapısı kullanılarak uygulanması planlanan birim test senaryoları Tablo 4.1'de tanımlanmıştır.

Table 4.1 Planlanan Fonksiyonel Test Senaryoları

Test ID	Test Edilecek Modül	Girdi / Aksiyon	Beklenen Davranış (Success Criteria)
T-01	Perspektif Geçişi	Oyuncu masaya oturur ve "E" tuşuna basar.	Kamera monitöre kilitlenmeli, FPS kontrolü kapanmalı, Mouse imleci aktifleşmelidir.
T-02	Raycast Sistemi	Oyuncu "Sunucu Kablosu" objesine bakar.	Ekranın ortasındaki imleç renk değiştirmeli ve "Kabloyu Bağla" ipucu metni belirmelidir.
T-03	Envanter/Veri	ScriptableObject veritabanına yeni bir soru eklenir.	Oyun başlatıldığında "Görev Yöneticisi" bu yeni soruyu listeye otomatik olarak çekmelidir.
T-04	Sınır Kontrolü	Karakter ofis duvarına doğru yürür.	"Mesh Collider" sistemi karakterin duvarın içinden geçmesini fiziksel olarak engellemelidir.

4.2. Uyumluluk ve Performans Hedefleri

Prototipin hedef platformda (Standart Ofis Bilgisayarı) sorunsuz çalışması için hedeflenen performans kriterleri şunlardır:

• Kare Hızı (FPS): Minimum 30 FPS, Hedef 60 FPS.

• Poligon Bütçesi: Low-Poly tasarım tercihi sayesinde sahne başına maksimum 50.000 üçgen (Triangle).

• Yükleme Süresi: Sahne geçişlerinde maksimum 5 saniye.

4.3. İçerik Geliştirme, Senaryolaştırma ve Doğrulama Süreci

Projenin temel amacı olan "mesleki deneyim kazandırma" hedefinin gerçekleşebilmesi için, oyun içerisindeki problemlerin rastgele değil, belirli bir pedagojik ve teknik kalite standardında olması gerekmektedir. Bu bağlamda içerik üretim süreci üç aşamalı bir kalite kontrol mekanizması ile kurgulanmıştır.

4.3.1. Problemlerin Akademik Yeterlilikleri (Müfredat Matrisi)

Oyun içeriği oluşturulurken Bilişim Sistemleri Mühendisliği lisans müfredatı temel alınmıştır. Soruların akademik geçerliliğini sağlamak adına bir "Müfredat-Problem Matrisi" oluşturulması planlanmıştır. Her bir oyun görevi, belirli bir dersin öğrenim çıktısı (Learning Outcome) ile eşleştirilecektir. Örnek: "Veritabanı Yönetimi" dersi "SQL Join Sorguları" konusu Oyun İçi Görev: "Satış Raporu Oluşturma".

Hazırlanan soru havuzu, sisteme entegre edilmeden önce ilgili alanın öğretim üyelerine (Örn: Ağ soruları için Ağ Yönetimi dersi hocasına) sunulacak; sorunun zorluk derecesi ve teknik doğruluğu onaylandıktan sonra veri eklenecektir.

4.3.2. Kurumsal ortama uygun senaryolaştırma (bağlamsal öğrenme)

Akademik soruların, oyunun akıcılığını (Flow) bozmadan simülasyona dahil edilmesi için "Hikayeleştirme" (Storytelling) tekniği kullanılmıştır. Ham Soru olarak bakarsak: "Aşağıdaki C# kodundaki sonsuz döngüyü bulunuz." (Klasik Sınav Sorusu) Senaryolaştırılmış görev ise: "Acil Durum! Maaş ödeme sistemi kilitlendi. Finans departmanı ödeme yapamıyor. Junior Mühendis, sisteme girip hatayı (bug) bul ve maaşlar yatmadan önce düzelt!" (Oyunlaştırılmış Görev) Bu yaklaşım sayesinde öğrenci sadece kodu düzeltmekle kalmaz; yazdığı kodun gerçek hayattaki etkisini (Maaşların yatması) ve zaman baskısını hissederek "Bağlamsal Öğrenme"yi gerçekleştirir

4.3.3. Unity entegrasyonu ve oyunlaştırma kalitesi

Hazırlanan senaryoların Unity oyun motoruna entegrasyonunda, kullanıcı deneyimini (UX) en üst düzeye çıkaracak arayüz tasarımları hedeflenmiştir. Diegetic (Oyun İçi) arayüzler, problemler ekrana yapıştırılmış bir metin kutusu (Pop-up) olarak değil, oyun dünyasının doğal bir parçası olarak sunulur. Oyuncu, soruyu görmek için sanal bilgisayarındaki "E-Posta" uygulamasını açmak veya sunucu odasındaki "Terminal Ekranına" bakmak zorundadır. Cevap verildiği anda sistem görsel ve işitsel geri bildirim sağlar. Başarılı durumda; teşil terminal ışıkları, "Sistem Onarıldı" sesi ve Kredi artışı animasyonu gözlenir. Başarısız durumda ise, Hata sirenleri, sunucudan duman çıkması efekti veya yöneticiden gelen uyarı mesajı gözlenebilir. Kalite ölçütü olarak bu görselleştirmeler, öğrencinin başarısızlık durumunda bile sıkılmadan tekrar denemesini (Replayability) sağlamayı amaçlar.

BÖLÜM 5. SONUÇLAR VE ÖNERİLER

5.1. Sonuçlar

Bu tasarım çalışması kapsamında, Bilişim Sistemleri Mühendisliği (BSM) öğrencilerinin mesleki yetkinliklerini sanal bir ortamda deneyimlemeleri için gerekli olan "Ciddi Oyun" (Serious Game) altyapısı (BSM-SİM) başarıyla tasarlanmış ve prototip aşamasına getirilmiştir.

Çalışma sonucunda elde edilen temel çıktılar ve kazanımlar şunlardır:

1. Modüler Veri Mimarisi: Unity oyun motoru üzerinde geliştirilen ScriptableObject tabanlı veri yapısı sayesinde, oyunun kaynak koduna müdahale edilmeden yeni akademik problemlerin (SQL soruları, Ağ senaryoları vb.) eklenebileceği sürdürülebilir bir sistem oluşturulmuştur.
2. Hibrit Deneyim: FPS (Gezinme) ve 2D Simülasyon (Masaüstü/Panel) mekanikleri, oluşturulan "Durum Makinesi" (State Machine) algoritmasıyla sorunsuz bir şekilde entegre edilmiş; böylece oyuncuya hem fiziksel ofis ortamı hem de teknik çalışma masası deneyimi bir arada sunulmuştur.
3. Ölçülebilir Başarı Modeli: "Saygınlık Kredisi" ve "Gün Döngüsü" formülasyonları ile öğrencinin soyut olan mesleki başarısı, oyun içinde somut ve sayısal verilere (Puan, Terfi, Kovulma) dönüştürülmüştür.
4. Faz-1 Doğrulaması: TÜBİTAK 2209-A projesi kapsamında hedeflenen "Teknik Prototip" fazı tamamlanmış ve proje, ikinci faz olan "Eğitsel Etkinlik Testleri"ne hazır hale getirilmiştir.

5.2. Öneriler ve Gelecek Çalışmalar

Tasarım aşaması tamamlanan BSM-SİM projesinin, nihai ürüne dönüştürülmesi ve yaygın etkisinin artırılması için aşağıdaki geliştirmeler önerilmektedir:

- Saha Testleri (Faz-2): Geliştirilen prototipin, Sakarya Üniversitesi BSM bölümü öğrencileri üzerinde test edilerek, oyunun akademik başarıya etkisinin nicel ve nitel yöntemlerle analiz edilmesi gerekmektedir.
- Yapay Zeka Destekli NPC: Mevcut diyalog ağaçlarının yerine, LLM (Büyük Dil Modelleri) tabanlı bir API entegre edilerek, oyun içi karakterlerin (Yönetici, İK Uzmanı) oyuncuya dinamik ve doğal yanıtlar vermesi sağlanabilir.
- VR (Sanal Gerçeklik) Adaptasyonu: Özellikle "Sunucu Odası" ve "Kablolama" görevlerinin VR gözlüklerle oynanabilir hale getirilmesi, "yaparak öğrenme" (hands-on learning) etkisini maksimuma çıkaracaktır.
- Analitik Dashboard**:** Öğretim üyelerinin, öğrencilerin oyun içindeki hatalarını toplu olarak görebileceği (Örn: "Sınıfın %60'ı SQL Join konusunda hata yaptı") bir web tabanlı izleme paneli geliştirilebilir.

KAYNAKLAR

[1] ARNAB, S., et al., Mapping learning and game mechanics for serious games analysis. British Journal of Educational Technology, 46(2):391-411, 2015.

[2] DE GLORIA, A., BELLOTTI, F., BERTA, R., LAVAGNINO, E., Serious Games for education and training. International Journal of Serious Games, 1(1), 2014.

[3] KOLB, D. A., Experiential learning: Experience as the source of learning and development. Prentice-Hall: Englewood Cliffs, NJ, 1984.

[4] MICHAEL, D. R., CHEN, S. L., Serious Games: Games That Educate, Train, and Inform. Muska & Lipman/Premier-Trade, 2005.

[5] NEGAHBAN, A., Simulation in engineering education: The transition from physical experimentation to digital immersive simulated environments. SIMULATION, 100(7):695–708, 2024.

[6] AKPINAR, Y., KUTBAY, E., Designing exploratory serious games with learning supports. Eğitimde Kuram ve Uygulama, 19(1):83-96, 2023..

[7] HAVERKAMP, V., HESSEL, V., LÖWE, H., MENGES, G., WARNIER MJF., REBROV, EV., de Croon MHJM, SCHOUTEN, JC., LIAUW, M., Hydrodynamics and mixer-induced bubble formation in microbubble columns with single and multiple channels. Chem. Eng. Technol., 29(9):1015–1026, 2006.

EKLER

intihal.net

Kaan_Biber_B231200382

Yazar : **Sayfa Sayısı :** 40 **Kelime Sayısı :** 4852 **Karakter Sayısı :** 36944

ORİJİNALLİK RAPORU

BENZERLİK ENDEKSİ	İNTERNET KAYNAKLARI	YAYINLAR	ÖĞRENCİ ÖDEVLERİ
%7	-	-	-

ÖZGEÇMİŞ

Kaan BİBER, 11/04/2002'de Rize/Pazar'da doğdu. İlk, orta ve lise eğitimini Rize'de tamamladı. 2020 yılında Pazar Fen Lisesi'nden mezun oldu. 2022 yılında Sakarya Üniversitesi Bilişim Sistemleri Mühendisliği Bölümü'nü kazandı. 2025 yılında Moral Demir firmasında işletme stajını yapmıştır. 2026 yılında Sakarya Üniversitesi Bilişim Sistemleri Mühendisliği Bölümü'nden mezun olma aşamasındadır.

ISE 401 BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ TASARIMI DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI

KONU : BSM-SİM: BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ ÖĞRENCİLERİNE YÖNELİK 3D BİLİŞİM İŞ SÜREÇLERİ SİMÜLASYON OYUNU (SERİOUS GAME)
ÖĞRENCİLER (Öğrenci No/AD/SOYAD): KAAN BİBER – B231200382

Değerlendirme Konusu	İstenenler	Not Aralığı	Not
Yazılı Çalışma			
Çalışma klavuza uygun olarak hazırlanmış mı?	x	0-5	
Teknik Yönden			
Problemin tanımı yapılmış mı?	x	0-5	
Geliştirilecek yazılımın/donanımın mimarisini içeren blok şeması (yazılımlar için veri akış şeması (dfd) da olabilir) çizilerek açıklanmış mı?			
Blok şemadaki birimler arasındaki bilgi akışına ait model/gösterim var mı?			
Yazılımın gereksinim listesi oluşturulmuş mu?			
Kullanılan/kullanılması düşünülen araçlar/teknolojiler anlatılmış mı?			
Donanımların programlanması/konfigürasyonu için yazılım gereksinimleri belirtilmiş mi?			
UML ile modelleme yapılmış mı?			
Veritabanları kullanılmış ise kavramsal model çıkarılmış mı? (Varlık ilişki modeli, noSQL kavramsal modelleri v.b.)			
Projeye yönelik iş-zaman çizelgesi çıkarılarak maliyet analizi yapılmış mı?			
Donanım bileşenlerinin maliyet analizi (prototip-adetli seri üretim vb.) çıkarılmış mı?			
Donanım için gerekli enerji analizi (minimum-uyku-aktif-maksimum) yapılmış mı?			
Grup çalışmalarında grup üyelerinin görev tanımları verilmiş mi (iş-zaman çizelgesinde belirtilebilir)?			
Sürüm denetim sistemi (Version Control System; Git, Subversion v.s.) kullanılmış mı?			
Sistemin genel testi için uygulanan metotlar ve iyileştirme süreçlerinin dökümü verilmiş mi?			
Yazılımın sızma testi yapılmış mı?			
Performans testi yapılmış mı?			
Tasarımın uygulamasında ortaya çıkan uyumsuzluklar ve aksaklıklar belirtilerek çözüm yöntemleri tartışılmış mı?			
Yapılan işlerin zorluk derecesi?	x	0-25	
Sözlü Sınav			
Yapılan sunum başarılı mı?	x	0-5	
Soruları yanıtlama yetkinliği?	x	0-20	
Devam Durumu			
Öğrenci dönem içerisindeki raporlarını düzenli olarak hazırladı mı?	x	0-5	
Diğer Maddeler			
Toplam			

DANIŞMAN : DR.ÖĞR.ÜYESİ BURCU ÇARKLI YAVUZ
DANIŞMAN İMZASI: